

Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: EJERCICIO 8

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Implementación y Código Fuente

El código fuente utiliza los mismos patrones que el ejercicio seis, pero reestructura el orden lógico de las declaraciones para evidenciar el mecanismo de resolución de ambigüedades del motor de Lex.

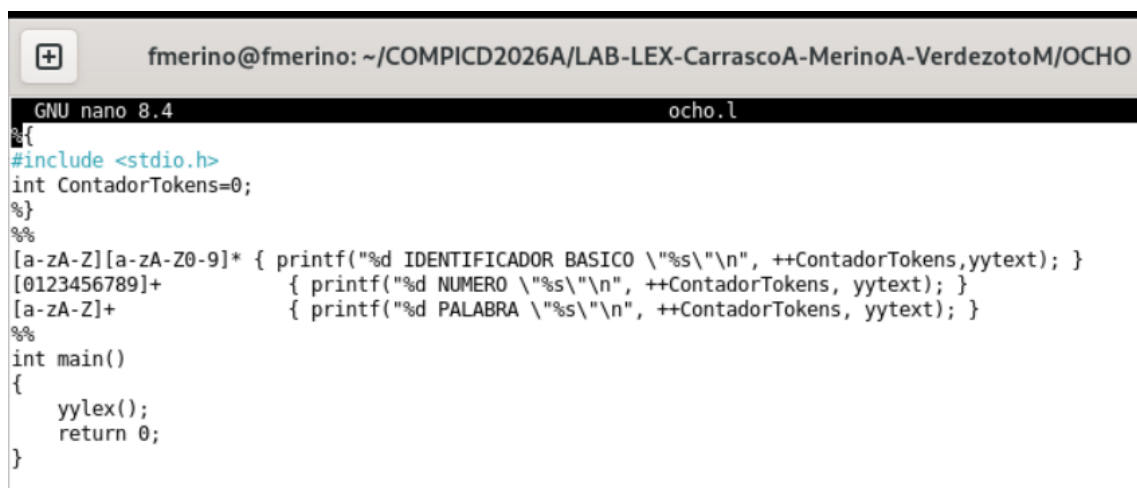
- [a-zA-Z][a-zA-Z0-9]*: Regla de identificadores (ubicada en la primera línea).
- [0123456789]+: Regla numérica (ubicada en la segunda línea).
- [a-zA-Z]+: Regla de palabras puras (ubicada en la tercera línea).

Tipos de entradas que recibe: Recibe secuencias alfanuméricas y numéricas puras.

Cadenas con las que el código fallaría (Errores Léxicos):

- Cadena compilador o x: Falla a nivel de lógica semántica. Aunque estas cadenas son "palabras" puras formadas solo por letras, el autómata las catalogará erróneamente como "IDENTIFICADOR BASICO" y nunca como "PALABRA". La regla de palabra se ha convertido en código inalcanzable (código muerto) debido a la prelación de la primera regla.

Código fuente del archivo ocho.l:



```
fmerino@fmerino: ~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO
GNU nano 8.4      ocho.l
%{
#include <stdio.h>
int ContadorTokens=0;
}%
%%
[a-zA-Z][a-zA-Z0-9]* { printf("%d IDENTIFICADOR BASICO \"%s\\n\"", ++ContadorTokens,yytext); }
[0123456789]+      { printf("%d NUMERO \"%s\\n\"", ++ContadorTokens, yytext); }
[a-zA-Z]+          { printf("%d PALABRA \"%s\\n\"", ++ContadorTokens, yytext); }
%%
int main()
{
    yylex();
    return 0;
}
```

Archivo entrada.txt:

```
fmerino@fmerino: ~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO
GNU nano 8.4 entrada.txt
1250
compilador
var123
99
x
```

2. Capturas de Ejecución

Se procesó un archivo de texto con múltiples sentencias que combinaban los mismos datos del ejercicio seis para observar la variación.

Ejecución final: Prueba de Procesamiento y Redirección (Consola y Archivo salida.txt):

```
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO$ lex ocho.l
ocho.l:8: warning, rule cannot be matched
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO$ gcc lex.yy.c -lfl
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO$ nano entrada.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO$ ./a.out < entrada.txt
1 NUMERO "1250"

2 IDENTIFICADOR BASICO "compilador"
3 IDENTIFICADOR BASICO "var123"
4 NUMERO "99"
5 IDENTIFICADOR BASICO "x"

fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO$ ./a.out < entrada.txt > salida.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/OCHO$ cat salida.txt
1 NUMERO "1250"

2 IDENTIFICADOR BASICO "compilador"
3 IDENTIFICADOR BASICO "var123"
4 NUMERO "99"
5 IDENTIFICADOR BASICO "x"
```

Descripción de lo más relevante de lo observado:

- **Cambio de Clasificación:** La cadena "compilador", que en el ejercicio previo era PALABRA, ahora es IDENTIFICADOR BASICO.
- **Resolución de Empates en Lex:** Se evidencia que cuando una cadena empata en longitud de coincidencia con dos reglas distintas, Lex ejecuta invariablemente la acción de la expresión regular que fue declarada primero en el archivo fuente de arriba hacia abajo.

3. Conclusión

El orden de declaración de las reglas es un factor crítico y determinante en la programación con Lex. Queda demostrado que los patrones más específicos o restrictivos (como palabras clave o símbolos únicos) deben declararse antes que los patrones genéricos (como los identificadores alfanuméricos) para prevenir el enmascaramiento y pérdida de tokens.